

FEM-NET Team Handoff

Project: Federated Early Misdiagnosis Network — privacy-preserving AI for women's health

Team: Kruthi (lead), Suraj, Kuber, Akshara

Stack: Python 3.13, PyTorch, Flower (FL), FastAPI, LangChain, FAISS, Vanilla JS

What Kruthi Has Built — Complete Overview

Kruthi built the entire system skeleton. Every file listed below is **done, tested for correctness, and must not be modified** unless you are explicitly fixing a bug.

Complete File Tree (Kruthi's work)

```

fem-net/
├── api/
│   ├── __init__.py
│   ├── main.py           ← FastAPI app entry point
│   ├── schemas.py        ← all Pydantic models (PatientInput, PredictionResponse, etc.)
│   └── routes/
│       ├── __init__.py
│       ├── predict.py     ← POST /api/predict
│       ├── insights.py    ← POST /api/insights
│       └── federated.py   ← GET/POST /api/federated/status + /run
├── federated/
│   ├── __init__.py
│   ├── strategy.py        ← KrumStrategy (Byzantine-robust FL aggregation)
│   ├── client.py          ← FemNetClient (Flower NumPyClient, local training)
│   └── server.py          ← Flower server launcher (for real network mode)
├── models/
│   ├── __init__.py
│   ├── base_model.py      ← ClinicalMLP architecture + get/set_parameters helpers
│   └── condition_models.py ← get_model("pcos") factory, save_model()
├── rag/
│   ├── __init__.py
│   ├── pubmed_fetcher.py  ← fetches real PubMed abstracts via BioPython Entrez
│   ├── embedder.py        ← FAISS index on SentenceTransformer embeddings
│   └── insight_generator.py ← LangChain + GPT-4o-mini → clinical insight card JSON
├── simulation/
│   ├── __init__.py
│   └── run_simulation.py   ← in-process FL sim using real processed CSVs
├── dashboard/
│   ├── index.html         ← clinician UI
│   └── static/
│       ├── style.css
│       └── app.js
├── data/
│   └── __init__.py        ← package marker only, no data generation code
├── .env.example
└── requirements.txt

```

How the System Works (read this fully)

TRAINING FLOW (Suraj + Kuber do this):

Real Kaggle CSV

- data/preprocess_<condition>.py
- data/processed/<condition>.csv ← standardised 10-feature + label CSV
- training/train_model.py
- models/weights/<condition>_global.pt ← PyTorch state_dict
- models/weights/<condition>_norm_stats.json ← {"age": {"mean": x, "std": y}, ...}

FEDERATED LEARNING FLOW (Kruthi's code, triggered after weights exist):

data/processed/<condition>.csv

- run_simulation.py splits into 3 non-IID hospital partitions
- FemNetClient (federated/client.py) trains locally on each partition
- weight updates (NOT raw data) sent to KrumStrategy
- KrumStrategy (federated/strategy.py) drops outliers, aggregates
- global model saved to models/weights/<condition>_global.pt ← OVERWRITES training weights
- simulation/results_<condition>.json saved

INFERENCE FLOW (runs live when API is up):

POST /api/predict (patient JSON)

- api/routes/predict.py loads models/weights/<condition>_global.pt
- loads models/weights/<condition>_norm_stats.json for normalisation
- ClinicalMLP.forward() → risk_score (0.0-1.0)
- returns {risk_score, risk_level, model_version}

POST /api/insights (patient JSON + risk_score)

- api/routes/insights.py
- rag/insight_generator.py queries FAISS index (real PubMed abstracts)
- GPT-4o-mini (or fallback) generates clinical insight card
- returns {risk_level, key_indicators, evidence_summary, recommended_next_steps}

The One Contract Everyone Must Respect

Every condition's processed CSV **must** have exactly these columns, in any order, with exactly these names:

```
age, bmi, fsh_level, lh_level, amh_level, tsh_level,  
cycle_length_days, follicle_count, fatigue_score, weight_gain_kg, label
```

- All values: float or int
- `label`: 0 or 1 only
- No NaN rows
- No extra columns (they are silently ignored but waste space)

The norm stats JSON **must** have exactly this structure:

```
{  
  "age":          {"mean": 34.2, "std": 8.1},  
  "bmi":          {"mean": 25.3, "std": 4.6},  
  "fsh_level":    {"mean": 7.1,  "std": 3.2},  
  "lh_level":     {"mean": 9.4,  "std": 5.1},  
  "amh_level":    {"mean": 3.2,  "std": 1.8},  
  "tsh_level":    {"mean": 2.5,  "std": 1.9},  
  "cycle_length_days": {"mean": 29.0, "std": 6.2},  
  "follicle_count": {"mean": 10.1, "std": 5.3},  
  "fatigue_score": {"mean": 5.0,  "std": 2.4},  
  "weight_gain_kg":  {"mean": 5.2,  "std": 4.1}  
}
```

Kruthi's `api/routes/predict.py` reads exactly this file at runtime. If it is missing or has wrong keys, normalisation silently skips and scores will be garbage.

SURAJ — Step-by-Step Instructions

Your deliverables:

```
data/processed/pcos.csv
data/processed/breast_cancer.csv
data/processed/cervical.csv
models/weights/pcos_global.pt
models/weights/breast_cancer_global.pt
models/weights/cervical_global.pt
models/weights/pcos_norm_stats.json
models/weights/breast_cancer_norm_stats.json
models/weights/cervical_norm_stats.json
training/train_model.py          ← shared trainer (Kuber uses the same file)
```

Step 0 — Clone repo and install deps

```
git clone <repo-url>
cd fem-net

# Install everything
pip install -r requirements.txt

# Install Kaggle CLI
pip install kaggle

# Set up Kaggle credentials
# Go to: kaggle.com → Your Account → API → Create New Token → downloads kaggle.json
mkdir -p ~/.kaggle
cp ~/Downloads/kaggle.json ~/.kaggle/
chmod 600 ~/.kaggle/kaggle.json
```

Step 1 — Create folders

```
mkdir -p data/raw/pcos data/raw/breast_cancer data/raw/cervical
mkdir -p data/processed
mkdir -p models/weights
mkdir -p training
touch training/__init__.py
```

Step 2 — Download datasets

```
kaggle datasets download praseekottarathil/polycystic-ovary-syndrome-pcos \
-p data/raw/pcos --unzip

kaggle datasets download shreyasvedpathak/pcos-dataset \
-p data/raw/pcos --unzip

kaggle datasets download uciml/breast-cancer-wisconsin-data \
-p data/raw/breast_cancer --unzip

kaggle datasets download ranzeet013/cervical-cancer-dataset \
-p data/raw/cervical --unzip

# Verify what downloaded
ls data/raw/pcos/
ls data/raw/breast_cancer/
ls data/raw/cervical/
```

Step 3 — Create `data/preprocess_pcos.py`

Before writing this file, run this first to see exact column names:

```
python3 -c "import pandas as pd; df=pd.read_csv('data/raw/pcos/PCOS_data_without_infer
```

Then write `data/preprocess_pcos.py` :

```

import pandas as pd
import numpy as np
import os

# --- Load both PCOS datasets ---
# Print columns first to verify names, then adjust the column_map below
files = [f for f in os.listdir("data/raw/pcos") if f.endswith(".csv")]
print("Files found:", files)

dfs = []
for f in files:
    try:
        dfs.append(pd.read_csv(f"data/raw/pcos/{f}"))
        print(f"Loaded {f}: {dfs[-1].shape}")
    except Exception as e:
        print(f"Skip {f}: {e}")

df = pd.concat(dfs, ignore_index=True)
print("\nAll columns:", df.columns.tolist())

# --- Map to FEM-NET schema ---
# ADJUST THESE NAMES to match what print(df.columns.tolist()) showed
column_map = {
    "Age (yrs)": "age",
    "BMI": "bmi",
    "FSH(mIU/mL)": "fsh_level",
    "LH(mIU/mL)": "lh_level",
    "AMH(ng/mL)": "amh_level",
    "TSH (mIU/L)": "tsh_level",
    "Cycle length(days)": "cycle_length_days",
    "Follicle No. (R)": "follicle_count",
    "Fatigue (Y/N)": "fatigue_score",
    "Weight gain(Y/N)": "weight_gain_kg",
    "PCOS (Y/N)": "label",
}

df = df.rename(columns=column_map)

FEATURES = ["age", "bmi", "fsh_level", "lh_level", "amh_level",
            "tsh_level", "cycle_length_days", "follicle_count",
            "fatigue_score", "weight_gain_kg"]

# Keep only columns that exist after renaming
available = [c for c in FEATURES if c in df.columns]
missing = [c for c in FEATURES if c not in df.columns]
if missing:
    print(f"WARNING: missing features {missing} - filling with median")
    for m in missing:
        df[m] = 0.0

# --- Clean ---
df = df[FEATURES + ["label"]].copy()
df = df.apply(pd.to_numeric, errors="coerce")
df["label"] = df["label"].fillna(0).astype(int)

```

```

df = df.dropna(thresh=int(len(df.columns) * 0.8))
df = df.fillna(df.median(numeric_only=True))
df = df[(df["label"] == 0) | (df["label"] == 1)]

os.makedirs("data/processed", exist_ok=True)
df[FEATURES + ["label"]].to_csv("data/processed/pcos.csv", index=False)
print(f"\nSaved: {len(df)} rows, {df['label'].mean():.1%} positive")
print(df[FEATURES].describe())

```

Run: `python3 data/preprocess_pcos.py`

Step 4 — Create `data/preprocess_breast_cancer.py`

```

import pandas as pd
import os

df = pd.read_csv("data/raw/breast_cancer/data.csv")
print("Columns:", df.columns.tolist())

FEATURES_SRC = [
    "radius_mean", "texture_mean", "perimeter_mean", "area_mean",
    "smoothness_mean", "compactness_mean", "concavity_mean",
    "concave points_mean", "symmetry_mean", "fractal_dimension_mean"
]

# Verify all exist
missing = [f for f in FEATURES_SRC if f not in df.columns]
if missing:
    print(f"Missing: {missing}. Available: {df.columns.tolist()}")
    raise SystemExit("Fix column names above before continuing.")

df["label"] = (df["diagnosis"] == "M").astype(int)
out = df[FEATURES_SRC + ["label"]].dropna().copy()

# Rename to FEM-NET standard names (positional mapping)
FEMNET_NAMES = ["age", "bmi", "fsh_level", "lh_level", "amh_level",
                 "tsh_level", "cycle_length_days", "follicle_count",
                 "fatigue_score", "weight_gain_kg"]
rename_map = dict(zip(FEATURES_SRC, FEMNET_NAMES))
out = out.rename(columns=rename_map)

os.makedirs("data/processed", exist_ok=True)
out.to_csv("data/processed/breast_cancer.csv", index=False)
print(f"Saved: {len(out)} rows, {out['label'].mean():.1%} malignant")
print(out.describe())

```

Run: `python3 data/preprocess_breast_cancer.py`

Step 5 — Create data/preprocess_cervical.py

```

import pandas as pd
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
import os

df = pd.read_csv("data/raw/cervical/cervical_cancer.csv")
print("Columns:", df.columns.tolist())
print("Shape:", df.shape)

df = df.replace("?", np.nan)
df = df.apply(pd.to_numeric, errors="coerce")

# Drop columns with >40% missing
thresh = len(df) * 0.6
df = df.dropna(thresh=thresh, axis=1)

# Target column — check what exists
for candidate in ["Biopsy", "biopsy", "Ca cervix", "dx"]:
    if candidate in df.columns:
        label_col = candidate
        break
else:
    print("Label column not found. Available:", df.columns.tolist())
    raise SystemExit

df = df.dropna(subset=[label_col])
df[label_col] = df[label_col].astype(int)

X = df.drop(columns=[label_col])
y = df[label_col]

X = X.fillna(X.median())

# Select top 10 features
k = min(10, X.shape[1])
selector = SelectKBest(f_classif, k=k)
X_sel = selector.fit_transform(X, y)
selected_names = X.columns[selector.get_support()].tolist()
print("Selected features:", selected_names)

FEMNET_NAMES = ["age", "bmi", "fsh_level", "lh_level", "amh_level",
                "tsh_level", "cycle_length_days", "follicle_count",
                "fatigue_score", "weight_gain_kg"]

out = pd.DataFrame(X_sel, columns=FEMNET_NAMES[:k])
# Pad missing columns with 0 if k < 10
for name in FEMNET_NAMES[k:]:
    out[name] = 0.0
out["label"] = y.values

os.makedirs("data/processed", exist_ok=True)

```

```
out.to_csv("data/processed/cervical.csv", index=False)
print(f"Saved: {len(out)} rows, {out['label'].mean():.1%} positive")
```

Run: `python3 data/preprocess_cervical.py`

Step 6 — Create `training/train_model.py`

This file is shared with Kuber. Write it once, both of you use it.

```

"""
FEM-NET unified trainer.
Reads: data/processed/<condition>.csv
Writes: models/weights/<condition>_global.pt
        models/weights/<condition>_norm_stats.json
"""

import argparse
import json
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, confusion_matrix

from models.base_model import ClinicalMLP

FEATURES = [
    "age", "bmi", "fsh_level", "lh_level", "amh_level",
    "tsh_level", "cycle_length_days", "follicle_count",
    "fatigue_score", "weight_gain_kg",
]

def train(condition: str, epochs: int = 50, lr: float = 3e-4, batch: int = 64):
    csv_path = f"data/processed/{condition}.csv"
    if not os.path.exists(csv_path):
        raise FileNotFoundError(f"Processed CSV not found: {csv_path}")

    df = pd.read_csv(csv_path)

    # Validate schema
    missing_cols = [c for c in FEATURES + ["label"] if c not in df.columns]
    if missing_cols:
        raise ValueError(f"CSV missing required columns: {missing_cols}")

    X = df[FEATURES].values.astype("float32")
    y = df["label"].values.astype("float32")

    # Compute and save norm stats BEFORE normalising
    mean = X.mean(axis=0)
    std = X.std(axis=0) + 1e-8

    os.makedirs("models/weights", exist_ok=True)
    norm_stats = {f: {"mean": float(mean[i]), "std": float(std[i])} for i, f in enumerate(FEATURES)}
    stats_path = f"models/weights/{condition}_norm_stats.json"
    with open(stats_path, "w") as f:
        json.dump(norm_stats, f, indent=2)

```

```

print(f"Norm stats saved → {stats_path}")

X = (X - mean) / std

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"Train: {len(X_train)} | Val: {len(X_val)} | Positive rate: {y.mean():.1%}")

loader_train = DataLoader(
    TensorDataset(torch.from_numpy(X_train), torch.from_numpy(y_train)),
    batch_size=batch, shuffle=True
)
loader_val = DataLoader(
    TensorDataset(torch.from_numpy(X_val), torch.from_numpy(y_val)),
    batch_size=batch
)

model = ClinicalMLP(input_dim=10)
opt = torch.optim.Adam(model.parameters(), lr=lr)
loss_fn = nn.BCELoss()

best_auc = 0.0
best_state = None

for epoch in range(1, epochs + 1):
    model.train()
    for xb, yb in loader_train:
        opt.zero_grad()
        loss_fn(model(xb), yb).backward()
        opt.step()

    model.eval()
    preds, labels = [], []
    with torch.no_grad():
        for xb, yb in loader_val:
            preds.extend(model(xb).numpy())
            labels.extend(yb.numpy())

    auc = roc_auc_score(labels, preds)
    if auc > best_auc:
        best_auc = auc
        best_state = {k: v.clone() for k, v in model.state_dict().items()}

    if epoch % 10 == 0:
        print(f"Epoch {epoch:3d} Val AUC: {auc:.4f} Best: {best_auc:.4f}")

# Save best checkpoint
weights_path = f"models/weights/{condition}_global.pt"
torch.save(best_state, weights_path)

# Final confusion matrix
model.load_state_dict(best_state)
model.eval()
with torch.no_grad():

```

```

        final_preds = (model(torch.from_numpy(X_val)).numpy() >= 0.5).astype(int)
        cm = confusion_matrix(y_val.astype(int), final_preds)

        print(f"\n{'='*40}")
        print(f"CONDITION : {condition.upper()}")
        print(f"Best AUC   : {best_auc:.4f}")
        print(f"Confusion matrix:\n{cm}")
        print(f"Weights    → {weights_path}")
        print(f"Norm stats → {stats_path}")
        print(f"{'='*40}\n")

if __name__ == "__main__":
    ALL = ["pcos", "breast_cancer", "cervical", "thyroid", "diabetes", "cardiovascular", "he
    parser = argparse.ArgumentParser()
    parser.add_argument("--condition", required=True, choices=ALL)
    parser.add_argument("--epochs", type=int, default=50)
    parser.add_argument("--lr", type=float, default=3e-4)
    parser.add_argument("--batch", type=int, default=64)
    args = parser.parse_args()
    train(args.condition, args.epochs, args.lr, args.batch)

```

Step 7 – Train your 3 conditions

```

python3 training/train_model.py --condition pcos --epochs 50
python3 training/train_model.py --condition breast_cancer --epochs 50
python3 training/train_model.py --condition cervical --epochs 50

```

Expected output for each: ``

**CONDITION : PCOS Best AUC : 0.83xx Confusion
matrix: [[TN FP] [FN TP]] Weights →**

models/weights/pcos_global.pt Norm stats→ models/weights/pcos_norm_stats.json

```

---

### Step 8 – Verify your outputs

```bash
Check files exist
ls models/weights/
Must show: pcos_global.pt, pcos_norm_stats.json
breast_cancer_global.pt, breast_cancer_norm_stats.json
cervical_global.pt, cervical_norm_stats.json

Validate norm stats format (Kruthi's API reads this)
python3 -c "
import json
for c in ['pcos','breast_cancer','cervical']:
 with open(f'models/weights/{c}_norm_stats.json') as f:
 s = json.load(f)
 required = ['age','bmi','fsh_level','lh_level','amh_level',
 'tsh_level','cycle_length_days','follicle_count',
 'fatigue_score','weight_gain_kg']
 missing = [k for k in required if k not in s]
 print(f'{c}: OK' if not missing else f'{c}: MISSING {missing}')
"

Validate CSV format (Kruthi's simulation reads this)
python3 -c "
import pandas as pd
for c in ['pcos','breast_cancer','cervical']:
 df = pd.read_csv(f'data/processed/{c}.csv')
 required = ['age','bmi','fsh_level','lh_level','amh_level',
 'tsh_level','cycle_length_days','follicle_count',
 'fatigue_score','weight_gain_kg','label']

```

```

missing = [c2 for c2 in required if c2 not in df.columns]
nulls = df.isnull().sum().sum()
print(f'{c}: {len(df)} rows, {df[\"label\"].mean():.1%} positive, nulls={nulls}, missi
"

```

---

## Step 9 – Git: what to commit

```

git add data/preprocess_pcos.py
git add data/preprocess_breast_cancer.py
git add data/preprocess_cervical.py
git add data/processed/pcos.csv
git add data/processed/breast_cancer.csv
git add data/processed/cervical.csv
git add training/train_model.py # shared with Kuber, coordinate
git add training/__init__.py
git add models/weights/pcos_global.pt
git add models/weights/pcos_norm_stats.json
git add models/weights/breast_cancer_global.pt
git add models/weights/breast_cancer_norm_stats.json
git add models/weights/cervical_global.pt
git add models/weights/cervical_norm_stats.json

git commit -m "feat: add PCOS/breast_cancer/cervical preprocessing, training, weights"

```

**DO NOT commit** `data/raw/` — those are large Kaggle downloads.

Add to `.gitignore`:

```

data/raw/
data/processed/_sim_*.csv
__pycache__/
*.pyc
.env

```

---



---

# KUBER — Step-by-Step Instructions

---

## Your deliverables:

```
data/processed/thyroid.csv
data/processed/diabetes.csv
data/processed/cardiovascular.csv
data/processed/heart.csv
models/weights/thyroid_global.pt
models/weights/diabetes_global.pt
models/weights/cardiovascular_global.pt
models/weights/heart_global.pt
models/weights/thyroid_norm_stats.json
models/weights/diabetes_norm_stats.json
models/weights/cardiovascular_norm_stats.json
models/weights/heart_norm_stats.json
```

You use the same `training/train_model.py` Suraj wrote. Coordinate with him — one of you writes it, push first, the other pulls.

---

## Step 0 — Setup (same as Suraj)

```
git clone <repo-url>
cd fem-net
pip install -r requirements.txt
pip install kaggle
mkdir -p ~/.kaggle && cp ~/Downloads/kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kag
mkdir -p data/raw/thyroid data/raw/diabetes data/raw/cardiovascular data/raw/heart
mkdir -p data/processed models/weights
```

## Step 1 – Download datasets

```
kaggle datasets download yasserhessein/thyroid-disease-data-set \
-p data/raw/thyroid --unzip

kaggle datasets download uciml/pima-indians-diabetes-database \
-p data/raw/diabetes --unzip

kaggle datasets download sulianova/cardiovascular-disease-dataset \
-p data/raw/cardiovascular --unzip

kaggle datasets download redwankarimsony/heart-disease-data \
-p data/raw/heart --unzip

ls data/raw/thyroid/
ls data/raw/diabetes/
ls data/raw/cardiovascular/
ls data/raw/heart/
```

---

**Step 2 — Create** data/preprocess\_thyroid.py

```

import pandas as pd
import numpy as np
import os

List all files to find the right one
print(os.listdir("data/raw/thyroid/"))

Thyroid dataset from Yasser Hessein has multiple CSV files
Load all and concat
dfs = []
for f in os.listdir("data/raw/thyroid/"):
 if f.endswith(".csv"):
 try:
 dfs.append(pd.read_csv(f"data/raw/thyroid/{f}"))
 print(f"Loaded {f}: {dfs[-1].shape}")
 print(dfs[-1].columns.tolist())
 except Exception as e:
 print(f"Skip {f}: {e}")

df = pd.concat(dfs, ignore_index=True) if dfs else pd.read_csv("data/raw/thyroid/thyro
print("\nFull columns:", df.columns.tolist())

--- Identify target column ---
Common names: 'binaryClass', 'Class', 'target', last column
for candidate in ["binaryClass", "Class", "class", "target"]:
 if candidate in df.columns:
 target_col = candidate
 break
else:
 target_col = df.columns[-1]
print(f"Using target column: {target_col}")
print(df[target_col].value_counts())

Binary: negative → 0, anything else → 1
df["label"] = (~df[target_col].astype(str).str.lower().str.contains("negative|n|-")).a

--- Continuous features ---
continuous_candidates = ["age", "TSH", "T3", "TT4", "T4U", "FTI"]
binary_candidates = ["on_thyroxine", "query_hypothyroid", "on_antithyroid_medication",

cont_avail = [c for c in continuous_candidates if c in df.columns]
bin_avail = [c for c in binary_candidates if c in df.columns]
print(f"Continuous available: {cont_avail}")
print(f"Binary available: {bin_avail}")

selected = (cont_avail + bin_avail)[:10]
Pad to 10 if needed
while len(selected) < 10:
 selected.append(selected[-1]) # duplicate last column temporarily
selected = selected[:10]

```

```
FEMNET_NAMES = ["age","bmi","fsh_level","lh_level","amh_level",
 "tsh_level","cycle_length_days","follicle_count",
 "fatigue_score","weight_gain_kg"]

out = df[selected + ["label"]].copy()
out.columns = FEMNET_NAMES + ["label"]
out = out.apply(pd.to_numeric, errors="coerce")
out = out.fillna(out.median(numeric_only=True))
out["label"] = out["label"].astype(int)
out = out[out["label"].isin([0, 1])]

os.makedirs("data/processed", exist_ok=True)
out.to_csv("data/processed/thyroid.csv", index=False)
print(f"Saved: {len(out)} rows, {out['label'].mean():.1%} positive")
```

Run: `python3 data/preprocess_thyroid.py`

---

**Step 3 — Create** data/preprocess\_diabetes.py

```

import pandas as pd
import numpy as np
import os

df = pd.read_csv("data/raw/diabetes/diabetes.csv")
print("Columns:", df.columns.tolist())
print(df.head(2))

Replace physiologically impossible zeros with NaN
zero_impossible = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
for col in zero_impossible:
 if col in df.columns:
 df[col] = df[col].replace(0, np.nan)
 df[col] = df[col].fillna(df[col].median())

Engineer 2 features to reach 10
df["glucose_bmi_ratio"] = df["Glucose"] / (df["BMI"] + 1e-8)
df["insulin_glucose_ratio"] = df["Insulin"] / (df["Glucose"] + 1e-8)

Map to FEM-NET schema
column_map = {
 "Pregnancies": "age",
 "Glucose": "bmi",
 "BloodPressure": "fsh_level",
 "SkinThickness": "lh_level",
 "Insulin": "amh_level",
 "BMI": "tsh_level",
 "DiabetesPedigreeFunction": "cycle_length_days",
 "Age": "follicle_count",
 "glucose_bmi_ratio": "fatigue_score",
 "insulin_glucose_ratio": "weight_gain_kg",
}
df = df.rename(columns=column_map)
df["label"] = df["Outcome"].astype(int)

FEMNET = ["age", "bmi", "fsh_level", "lh_level", "amh_level",
 "tsh_level", "cycle_length_days", "follicle_count",
 "fatigue_score", "weight_gain_kg"]

os.makedirs("data/processed", exist_ok=True)
df[FEMNET + ["label"]].dropna().to_csv("data/processed/diabetes.csv", index=False)
print(f"Saved: {len(df)} rows, {df['label'].mean():.1%} positive")

```

Run: python3 data/preprocess\_diabetes.py

**Step 4 — Create** data/preprocess\_cardiovascular.py

```

import pandas as pd
import numpy as np
import os

Note: this dataset uses semicolons as separator
df = pd.read_csv("data/raw/cardiovascular/cardio_train.csv", sep=";")
print("Columns:", df.columns.tolist())
print(f"Shape: {df.shape}")

Engineer features
df["age_years"] = (df["age"] / 365).round(1)
df["bmi_calc"] = df["weight"] / ((df["height"] / 100) ** 2)

Remove outlier blood pressure readings
df = df[(df["ap_hi"].between(60, 250)) & (df["ap_lo"].between(40, 160))]

column_map = {
 "age_years": "age",
 "bmi_calc": "bmi",
 "ap_hi": "fsh_level",
 "ap_lo": "lh_level",
 "cholesterol": "amh_level",
 "gluc": "tsh_level",
 "smoke": "cycle_length_days",
 "alco": "follicle_count",
 "active": "fatigue_score",
 "gender": "weight_gain_kg",
}

df = df.rename(columns=column_map)
df["label"] = df["cardio"].astype(int)

FEMNET = ["age", "bmi", "fsh_level", "lh_level", "amh_level",
 "tsh_level", "cycle_length_days", "follicle_count",
 "fatigue_score", "weight_gain_kg"]

os.makedirs("data/processed", exist_ok=True)
df[FEMNET + ["label"]].dropna().to_csv("data/processed/cardiovascular.csv", index=False)
print(f"Saved: {len(df)} rows, {df['label'].mean():.1%} positive")

```

Run: python3 data/preprocess\_cardiovascular.py



**Step 5 — Create** data/preprocess\_heart.py

```

import pandas as pd
import numpy as np
import os

print(os.listdir("data/raw/heart/"))
df = pd.read_csv("data/raw/heart/heart_disease_uci.csv") # adjust filename if differe
print("Columns:", df.columns.tolist())
print(df.head(2))

Standard Cleveland features (adjust names if different)
column_map = {
 "age": "age",
 "sex": "bmi",
 "cp": "fsh_level",
 "trestbps": "lh_level",
 "chol": "amh_level",
 "fbs": "tsh_level",
 "restecg": "cycle_length_days",
 "thalach": "follicle_count",
 "exang": "fatigue_score",
 "oldpeak": "weight_gain_kg",
}

Check which source columns exist
for src, dst in list(column_map.items()):
 if src not in df.columns:
 print(f"WARNING: '{src}' not found. Available: {df.columns.tolist()}")

df = df.rename(columns=column_map)

Target: 'num' column (0=no disease, 1-4=disease) – binarise
for candidate in ["num", "target", "condition", "heart_disease"]:
 if candidate in df.columns:
 df["label"] = (df[candidate] > 0).astype(int)
 break
else:
 print("Target column not found. Available:", df.columns.tolist())
 raise SystemExit

FEMNET = ["age", "bmi", "fsh_level", "lh_level", "amh_level",
 "tsh_level", "cycle_length_days", "follicle_count",
 "fatigue_score", "weight_gain_kg"]

os.makedirs("data/processed", exist_ok=True)
df[FEMNET + ["label"]].dropna().to_csv("data/processed/heart.csv", index=False)
print(f"Saved: {len(df)} rows, {df['label'].mean():.1%} positive")

```

Run: python3 data/preprocess\_heart.py

**Step 6 — Get `training/train_model.py` from Suraj**

```
git pull # get Suraj's committed train_model.py
```

If Suraj hasn't pushed yet, write it from the code in Suraj's Step 6 above (identical file).

---

**Step 7 — Train your 4 conditions**

```
python3 training/train_model.py --condition thyroid --epochs 50
python3 training/train_model.py --condition diabetes --epochs 50
python3 training/train_model.py --condition cardiovascular --epochs 50
python3 training/train_model.py --condition heart --epochs 50
```

Target AUCs:

- thyroid: > 0.85
  - diabetes: > 0.82
  - cardiovascular: > 0.79
  - heart: > 0.88
- 

**Step 8 — Verify (same validation script as Suraj, different conditions)**

```
python3 -c "
import json
for c in ['thyroid','diabetes','cardiovascular','heart']:
 with open(f'models/weights/{c}_norm_stats.json') as f:
 s = json.load(f)
 required = ['age','bmi','fsh_level','lh_level','amh_level',
 'tsh_level','cycle_length_days','follicle_count',
 'fatigue_score','weight_gain_kg']
 missing = [k for k in required if k not in s]
 print(f'{c}: OK' if not missing else f'{c}: MISSING {missing}')
"
```

---

## Step 9 – Git: what to commit

```
git add data/preprocess_thyroid.py data/preprocess_diabetes.py
git add data/preprocess_cardiovascular.py data/preprocess_heart.py
git add data/processed/thyroid.csv data/processed/diabetes.csv
git add data/processed/cardiovascular.csv data/processed/heart.csv
git add models/weights/thyroid_global.pt models/weights/thyroid_norm_stats.json
git add models/weights/diabetes_global.pt models/weights/diabetes_norm_stats.json
git add models/weights/cardiovascular_global.pt models/weights/cardiovascular_norm_stats.json
git add models/weights/heart_global.pt models/weights/heart_norm_stats.json

git commit -m "feat: add thyroid/diabetes/cardiovascular/heart preprocessing, training"
```

# AKSHARA — Step-by-Step Instructions

---

## Your deliverables:

```
data/dataset_registry.json
dashboard/index.html ← updated (expanded dropdown + provenance card)
dashboard/static/app.js ← updated (provenance logic)
data/processed/endometriosis.csv ← if dataset found
```

---

## Step 0 — Setup

```
git clone <repo-url>
cd fem-net
pip install -r requirements.txt
```

---

## Step 1 — Search for Endometriosis / missing women's health datasets

Search these in this order:

### Kaggle:

```
https://www.kaggle.com/search?q=endometriosis
https://www.kaggle.com/search?q=menstrual+disorder
https://www.kaggle.com/search?q=ovarian+cancer+clinical
https://www.kaggle.com/search?q=PCOS+endometriosis
```

### Other sources:

- <https://physionet.org/search/#query=endometriosis>
- <https://data.mendeley.com/> → search "endometriosis"
- <https://dataverse.harvard.edu/> → search "women health clinical"
- <https://archive.ics.uci.edu/datasets> → search "gynecology"

**For each dataset found, record:**

1. Name
  2. Exact URL
  3. Row count
  4. Column/feature list
  5. Label column name
  6. License (must be CC0, CC-BY, or public domain to use)
  7. Usable for FEM-NET? Yes/No + reason
- 

**Step 2 — Write** `data/dataset_registry.json`

Create this file. Fill in row counts from Suraj/Kuber's preprocessing output:

```

{
 "pcos": {
 "source": "kaggle:prasoonkottarathil/polycystic-ovary-syndrome-pcos",
 "secondary_source": "kaggle:shreyasvedpathak/pcos-dataset",
 "rows": 0,
 "positive_rate": 0.0,
 "features": ["age", "bmi", "fsh_level", "lh_level", "amh_level", "tsh_level", "cycle_len", "label_column_original": "PCOS (Y/N)",
 "license": "CC0",
 "status": "trained",
 "assigned_to": "Suraj"
 },
 "breast_cancer": {
 "source": "kaggle:uciml/breast-cancer-wisconsin-data",
 "rows": 569,
 "positive_rate": 0.373,
 "features": ["radius_mean", "texture_mean", "perimeter_mean", "area_mean", "smoothness", "label_column_original": "diagnosis (M=1, B=0)",
 "license": "CC0",
 "status": "trained",
 "assigned_to": "Suraj"
 },
 "cervical": {
 "source": "kaggle:ranzeet013/cervical-cancer-dataset",
 "rows": 858,
 "positive_rate": 0.055,
 "features": "top 10 by SelectKBest from original 36",
 "label_column_original": "Biopsy",
 "license": "CC BY 4.0",
 "status": "trained",
 "assigned_to": "Suraj"
 },
 "thyroid": {
 "source": "kaggle:yasserhessein/thyroid-disease-data-set",
 "rows": 0,
 "positive_rate": 0.0,
 "label_column_original": "binaryClass",
 "license": "Public",
 "status": "trained",
 "assigned_to": "Kuber"
 },
 "diabetes": {
 "source": "kaggle:uciml/pima-indians-diabetes-database",
 "rows": 768,
 "positive_rate": 0.349,
 "label_column_original": "Outcome",
 "license": "CC0",
 "status": "trained",
 "assigned_to": "Kuber"
 },
 "cardiovascular": {
 "source": "kaggle:sulianova/cardiovascular-disease-dataset",
 "rows": 70000,
 "positive_rate": 0.499,

```

```

 "label_column_original": "cardio",
 "license": "CC BY 4.0",
 "status": "trained",
 "assigned_to": "Kuber"
 },
 "heart": {
 "source": "kaggle:redwankarimsony/heart-disease-data",
 "rows": 920,
 "positive_rate": 0.0,
 "label_column_original": "num (binarised > 0)",
 "license": "CC BY 4.0",
 "status": "trained",
 "assigned_to": "Kuber"
 },
 "endometriosis": {
 "source": "TBD",
 "status": "searching",
 "assigned_to": "Akshara"
 }
}

```

Fill in the `rows` and `positive_rate` values after Suraj/Kuber push their preprocessing scripts.

---

### Step 3 — Update `dashboard/index.html` — expand condition dropdown

Open `dashboard/index.html`. Find this block (around line 70):

```

<select name="condition">
 <option value="pcos">PCOS</option>
 <option value="endometriosis">Endometriosis</option>
 <option value="thyroid">Thyroid Disorder</option>
</select>

```

Replace with:

```
<select name="condition" id="condition-select">
 <option value="pcos">PCOS</option>
 <option value="breast_cancer">Breast Cancer</option>
 <option value="cervical">Cervical Cancer</option>
 <option value="thyroid">Thyroid Disorder</option>
 <option value="diabetes">Diabetes</option>
 <option value="cardiovascular">Cardiovascular Disease</option>
 <option value="heart">Heart Disease</option>
</select>
```

#### Step 4 — Add provenance card to `dashboard/index.html`

Find the closing `</div>` of the risk assessment card (the one with id `gauge-result`). Add this block immediately **after** that card's closing `</div>`:

```
<!-- Dataset Provenance Card -->
<div class="card" id="provenance-card" style="display:none; margin-bottom:24px">
 <h2>
 <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
 <path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10z"/>
 </svg>
 Dataset Provenance
 </h2>
 <div id="provenance-name" style="font-weight:700;font-size:0.9rem;margin-bottom:6px">
 <div id="provenance-meta" style="font-size:0.82rem;color:#6b7280;margin-bottom:6px">
 <div id="provenance-note" style="font-size:0.82rem;color:#4b5563;border-left:3px solid #4b5563">
</div>
```

#### Step 5 — Add provenance logic to `dashboard/static/app.js`

Open `dashboard/static/app.js`. At the very top after `'use strict';`, add:



```

const DATASET_INFO = {
 pcos: {
 name: "PCOS Dataset – Prasoon Kottarathil + Shreyas Vedpathak",
 meta: "Kaggle · CC0 · ~541 rows merged",
 note: "LH/AMH ratios, follicle count, cycle length – primary PCOS markers"
 },
 breast_cancer: {
 name: "Breast Cancer Wisconsin (Diagnostic) – UCI",
 meta: "Kaggle · CC0 · 569 rows",
 note: "Cell nucleus measurements (radius, texture, perimeter). Top 10 means select"
 },
 cervical: {
 name: "Cervical Cancer Risk Factors – Ranzeet013",
 meta: "Kaggle · CC BY 4.0 · 858 rows",
 note: "Biopsy as ground truth. Top 10 features selected via SelectKBest (f_classif"
 },
 thyroid: {
 name: "Thyroid Disease Dataset – Yasser Hessein",
 meta: "Kaggle · Public · 9,172 rows",
 note: "TSH, T3, T4, FTI levels. Binary: hypothyroid vs negative."
 },
 diabetes: {
 name: "Pima Indians Diabetes Database – UCI",
 meta: "Kaggle · CC0 · 768 rows",
 note: "Female patients ≥21 years. Glucose, insulin, BMI, diabetes pedigree functio"
 },
 cardiovascular: {
 name: "Cardiovascular Disease Dataset – Sulianova",
 meta: "Kaggle · CC BY 4.0 · 70,000 rows",
 note: "Systolic/diastolic BP, cholesterol, glucose, BMI. Largest dataset in FEM-NE"
 },
 heart: {
 name: "Heart Disease UCI – Redwan Karim Sony",
 meta: "Kaggle · CC BY 4.0 · 920 rows",
 note: "Cleveland + Hungary + VA + Switzerland combined. Target column binarised (n"
 }
};

function updateProvenance() {
 const condition = document.querySelector('select[name="condition"]').value;
 const info = DATASET_INFO[condition];
 const card = document.getElementById('provenance-card');
 if (!info) { card.style.display = 'none'; return; }
 document.getElementById('provenance-name').textContent = info.name;
 document.getElementById('provenance-meta').textContent = info.meta;
 document.getElementById('provenance-note').textContent = info.note;
 card.style.display = 'block';
}

```

Then in the `DOMContentLoaded` handler (bottom of `app.js`), add these two lines:

```
document.querySelector('select[name="condition"]').addEventListener('change', updatePr
updateProvenance());
```

## Step 6 — If you find an endometriosis dataset

If you find a usable dataset (CC0 or CC-BY license, >200 rows, has diagnosis label):

```
Download it
kaggle datasets download <slug> -p data/raw/endometriosis --unzip

Write data/preprocess_endometriosis.py
Follow exact same pattern as Suraj's preprocess scripts
Output must be: data/processed/endometriosis.csv
Must have columns: age,bmi,fsh_level,lh_level,amh_level,tsh_level,
cycle_length_days,follicle_count,fatigue_score,weight_gain_kg,lab

Train it
python3 training/train_model.py --condition endometriosis --epochs 50
```

Then tell Kruthi — she will add "endometriosis" to:

- `models/condition_models.py` SUPPORTED set
- `api/schemas.py` Literal
- `api/main.py` ALL\_CONDITIONS list
- `simulation/run_simulation.py` ALL\_CONDITIONS list

## Step 7 — Git: what to commit

```
git add data/dataset_registry.json
git add dashboard/index.html
git add dashboard/static/app.js
If endometriosis found:
git add data/preprocess_endometriosis.py
git add data/processed/endometriosis.csv
git add models/weights/endometriosis_global.pt
git add models/weights/endometriosis_norm_stats.json

git commit -m "feat: dataset registry, expanded dashboard conditions, provenance cards"
```


## **Final Integration Checklist (All 4 Together)**

---

Run this after everyone has pushed:

```

git pull # get everyone's work

1. Check all weights and norm stats exist
ls models/weights/
Expected: 7x _global.pt + 7x _norm_stats.json

2. Check all processed CSVs exist
ls data/processed/*.csv
Expected: pcos.csv breast_cancer.csv cervical.csv thyroid.csv diabetes.csv cardiovas

3. Validate all CSVs match schema
python3 -c "
import pandas as pd, json, os
FEATURES = ['age', 'bmi', 'fsh_level', 'lh_level', 'amh_level', 'tsh_level',
 'cycle_length_days', 'follicle_count', 'fatigue_score', 'weight_gain_kg']
for c in ['pcos', 'breast_cancer', 'cervical', 'thyroid', 'diabetes', 'cardiovascular', 'hea
df = pd.read_csv(f'data/processed/{c}.csv')
bad = [f for f in FEATURES+['label'] if f not in df.columns]
nulls = df.isnull().sum().sum()
print(f'{c:20s} rows={len(df):6d} pos={df[\"label\"].mean():.1%} nulls={nulls}
"

4. Run FL simulation end-to-end
python3 -m simulation.run_simulation --condition pcos --rounds 5

5. Start API
uvicorn api.main:app --reload

6. Test predict endpoint
curl -s -X POST http://localhost:8000/api/predict \
-H "Content-Type: application/json" \
-d '{"age":28,"bmi":24.5,"fsh_level":7.2,"lh_level":12.4,"amh_level":3.1,
 "tsh_level":2.5,"cycle_length_days":30,"follicle_count":10,
 "fatigue_score":5,"weight_gain_kg":3.0,"condition":"pcos"}' | python3 -m json.t

7. Test all conditions
for COND in pcos breast_cancer cervical thyroid diabetes cardiovascular heart; do
echo "Testing $COND..."
curl -s -X POST http://localhost:8000/api/predict \
-H "Content-Type: application/json" \
-d '{"age":35,"bmi":27,"fsh_level":8,"lh_level":10,"amh_level":2,
 "tsh_level":3,"cycle_length_days":28,"follicle_count":8,
 "fatigue_score":6,"weight_gain_kg":4,"condition":"$COND"}' \
| python3 -m json.tool
done

8. Open browser → http://localhost:8000
Change condition dropdown → provenance card updates
Fill form → submit → gauge animates → insight card appears

```

## Conflict Rules

FILE	OWNER	OTHERS MAY
federated/strategy.py	Kruthi	read only
federated/client.py	Kruthi	read only
federated/server.py	Kruthi	read only
simulation/run_simulation.py	Kruthi	read only
models/base_model.py	Kruthi	read only
models/condition_models.py	Kruthi	read only
api/main.py	Kruthi	read only
api/schemas.py	Kruthi	read only
api/routes/*.py	Kruthi	read only
rag/*.py	Kruthi	read only
dashboard/index.html	Akshara	edit (Steps 3-4)
dashboard/static/app.js	Akshara	edit (Step 5)
dashboard/static/style.css	Kruthi	read only
data/preprocess_pcos.py	Suraj	owns
data/preprocess_breast_cancer.py	Suraj	owns
data/preprocess_cervical.py	Suraj	owns
data/preprocess_thyroid.py	Kuber	owns
data/preprocess_diabetes.py	Kuber	owns
data/preprocess_cardiovascular.py	Kuber	owns
data/preprocess_heart.py	Kuber	owns
training/train_model.py	Suraj writes first, Kuber uses	coordinate push order

data/processed/*.csv	Suraj (pcos/breast/cervical), Kuber (rest)	owns per condition
models/weights/*.pt	Suraj (pcos/breast/cervical), Kuber (rest)	owns per condition
data/dataset_registry.json	Akshara	owns
requirements.txt	Kruthi	do not modify
.gitignore	anyone	add to it, never remove lines